EJERCICIO 3 - Estructuras del grafo, Floyd, centro del grafo y Warshall

Grafo: ver ej3_grafo.png (cargado desde aeropuertos.txt y conexiones.txt).

En los órdenes, V es la cantidad de vértices y E la cantidad de aristas.

---

1. ESTRUCTURAS DE DATOS

El grafo se guarda como lista de adyacencias en un solo mapa:

```
    Map<V, List<Edge<V, D>>> adyacencias = new LinkedHashMap<>();
```

- La clave es el vértice (el aeropuerto) y el valor es la lista de sus aristas salientes. Cada arista guarda origen, destino y un WeightedEdge con el peso (la distancia).

Por qué cada estructura:
- Map: no permite vértices repetidos y da la lista de un vértice en O(1).
- LinkedHashMap (y no HashMap): mantiene el orden de inserción, así los recorridos y las matrices salen en el orden del archivo y siempre igual. TreeMap obligaría a que V sea Comparable y cuesta O(log V).
- ArrayList para las adyacencias: agregar al final es O(1) y recorrerla es O(grado). Ocupa menos memoria que una LinkedList.
- Set para vertices(), sucesores y predecesores: no hay repetidos. vertices() devuelve el conjunto de claves del mapa, sin copiarlo (O(1)); sucesores y predecesores arman un conjunto nuevo en cada llamada.
- Lista de adyacencias y no matriz: ocupa O(V + E) en vez de O(V²). La red es dispersa (24 vuelos de 56 posibles). Floyd y Warshall arman su propia matriz al empezar, que es donde la matriz conviene.

Órdenes en esta implementación:
- agregarVertice: O(1).
- buscarVertice: O(V), porque se busca con un Comparable y no con la clave.
- agregarArista, existeArista, adyacencias: O(V + grado).
- predecessors: O(V + E), porque no se guardan las aristas entrantes.
- Memoria: O(V + E).

agregarVertice y agregarArista ya estaban implementados. Devuelven falso y no cambian nada si el vértice ya existe, o si la arista ya existe o le falta alguno de sus vértices.

NOTA: los archivos no coinciden con la Tabla 2 de la letra (en la tabla Porto Alegre no tiene vuelos salientes, Montevideo -> San Pablo cuesta 980 y no 2000, y está Montevideo -> Santos 1200). Se usaron los archivos, como pide la letra.

---

2. FLOYD

Lenguaje natural:
Se arma una matriz A con los costos de los arcos: 0 en la diagonal e infinito donde no hay vuelo. En la iteración k se permite usar el vértice k como escala: si ir de i a k y de k a j es más barato que A[i, j], se actualiza A[i, j] y se guarda k en P[i, j]. Al terminar, A tiene los costos mínimos y P sirve para recuperar los caminos.

Precondiciones: pesos no negativos.

Postcondiciones:
- El grafo no cambia.
- A[i, j] es el menor costo de i a j (infinito si no hay camino).
- P[i, j] es el intermedio que dio ese costo, o nulo si es el arco directo.

Pseudocódigo:

```
floyd(grafo) devuelve (A, P)
Comienzo
    PARA i <- 1 HASTA n HACER
        PARA j <- 1 HASTA n HACER
            SI i = j ENTONCES A[i, j] <- 0 SINO A[i, j] <- infinito FIN SI
            P[i, j] <- nulo
        FIN PARA
        PARA CADA arco EN aristasSalientes(i) HACER
            A[i, destino(arco)] <- peso(arco)
        FIN PARA
    FIN PARA
    PARA k <- 1 HASTA n HACER
        PARA i <- 1 HASTA n HACER
            PARA j <- 1 HASTA n HACER
                SI A[i, k] + A[k, j] < A[i, j] ENTONCES
                    A[i, j] <- A[i, k] + A[k, j]
                    P[i, j] <- k
                FIN SI
            FIN PARA
        FIN PARA
    FIN PARA
    devolver (A, P)
Fin
```

Recuperar el camino de i a j (se agrega i al principio y j al final):

```
intermedios(i, j, camino)
Comienzo
    k <- P[i, j]
    SI k <> nulo ENTONCES
        intermedios(i, k, camino)
        camino.agregarAlFinal(k)
        intermedios(k, j, camino)
    FIN SI
Fin
```

Orden del tiempo de ejecución:
Los tres PARA anidados hacen $O(V^3)$. La memoria es $O(V^2)$ por las matrices.
Recuperar un camino es $O(V)$.

Resultado (matriz A):

	Asu	BA	Cur	Mvd	PoA	Rio	SP	Sts
Asu	0	1600	800	1700	700	2300	1680	1900
BA	1600	0	2400	200	1200	3900	2180	2400
Cur	4500	5480	0	5280	4280	1500	3300	3400
Mvd	1800	200	2580	0	1000	3780	1980	2200
PoA	2180	1200	1580	1000	0	2780	980	1200
Rio	3000	3980	1500	3780	2780	0	1800	1900
SP	1200	2180	600	1980	980	1800	0	220

```
    Sts  1420 2400   820  2200  1200  2020   220     0
```

Ejemplos:
- Montevideo -> Rio de Janeiro: 3780,
  por Montevideo -> Porto Alegre -> San Pablo -> Rio de Janeiro.
- Curitiba -> Montevideo: 5280,
  por Curitiba -> Rio de Janeiro -> San Pablo -> Porto Alegre -> Montevideo.

---

3. CENTRO DEL GRAFO (CENTRO DE MANTENIMIENTO)

Lenguaje natural:
La excentricidad de un vértice v es la mayor distancia mínima desde los otros vértices hacia v, o sea, el máximo de la columna de v en la matriz A. El centro es el vértice de menor excentricidad: el aeropuerto al que se llega desde cualquier otro con el peor caso más barato.

Precondiciones: la de Floyd.

Postcondiciones: devuelve el vértice de menor excentricidad (nulo si el grafo está vacío). El grafo no cambia.

Pseudocódigo:

```
centro(grafo) devuelve Vertice
Comienzo
    A <- floyd(grafo)
    centro <- nulo
    menor <- infinito
    PARA CADA v EN vertices(grafo) HACER
        e <- máximo de A[u, v] para todo u <> v
        SI centro = nulo O e < menor ENTONCES
            centro <- v
            menor <- e
        FIN SI
    FIN PARA
    devolver centro
Fin
```

Orden del tiempo de ejecución:
Floyd es $O(V^3)$ y recorrer la matriz es $O(V^2)$. Total: $O(V^3)$.

Resultado (excentricidades):
    Asuncion 4500, Buenos_Aires 5480, Curitiba 2580, Montevideo 5280,
    Porto_Alegre 4280, Rio_de_Janeiro 3900, San_Pablo 3300, Santos 3400.

El aeropuerto más apropiado para el centro de mantenimiento es CURITIBA.
Desde cualquier aeropuerto se llega a Curitiba con 2580 como máximo
(el peor caso es desde Montevideo).

---

4. WARSHALL

Lenguaje natural:
Solo interesa saber si hay camino de i a j, no su costo. Se arma una matriz booleana A con verdadero donde hay arco. En la iteración k, si todavía no

hay camino de i a j pero hay de i a k y de k a j, ahora sí lo hay.

Precondiciones: la común (el grafo está creado).

Postcondiciones:
- El grafo no cambia.
- A[i, j] es verdadero si y solo si hay camino de i a j.
- A[i, i] es verdadero solo si i está en un ciclo.

Pseudocódigo:

```
warshall(grafo) devuelve A
Comienzo
    PARA i <- 1 HASTA n HACER
        PARA j <- 1 HASTA n HACER
            A[i, j] <- existeArista(i, j)
        FIN PARA
    FIN PARA
    PARA k <- 1 HASTA n HACER
        PARA i <- 1 HASTA n HACER
            PARA j <- 1 HASTA n HACER
                SI NO A[i, j] Y A[i, k] Y A[k, j] ENTONCES
                    A[i, j] <- VERDADERO
                FIN SI
            FIN PARA
        FIN PARA
    FIN PARA
    devolver A
Fin
```

Orden del tiempo de ejecución: $O(V^3)$, igual que Floyd.

Resultado: la matriz da verdadero en todas las posiciones, incluida la diagonal. Desde cualquier aeropuerto se llega a cualquier otro, y todos están en algún ciclo.